מדריך למידה מרוכז · גרסת ספטמבר 2026

למידה חישובית וכריית נתונים

סיכום מלא ומסודר של חומר הקורס — מהיסודות ועד SVM, AdaBoost ו־Clustering

מבוסס רק על סיכומי הקורס שנבדקו מול מצגות SHAY ML.

המסמך אינו כולל חומר חיצוני. Naive Bayes הושמט משום שאינו בחומר הבחינה שסומן במפת הקורס.

איך להשתמש במסמך

קריאה ראשונה: לעבור לפי סדר הפרקים ולהבין את הרעיון. **חזרה:** להשתמש ב"מה חייבים לדעת" ובדף הנוסחאות.
תרגול: לפתור לפי תבניות השלבים, בלי לדלג ישר לתוצאה.

תוכן עניינים

1. יסודות למידה חישובית והערכת ביצועים 1
2. K-Nearest Neighbors 2
3. עצי החלטה ו־ID3 3
4. Perceptron ורשתות נוירונים 4
5. Support Vector Machine 5
6. Ensemble Learning ו־AdaBoost 6
7. Cluster Analysis 7
8. השוואת האלגוריתמים 8
9. דף נוסחאות ותבניות פתרון 9
10. צ׳קליסט למבחן 10

מונחים מקצועיים נשמרו באנגלית לצד ההסבר בעברית, כדי להתאים לניסוח המצגות והשאלות.

יסודות למידה חישובית

הרעיון המרכזי: מערכת לומדת מניסיון אם הביצועים שלה במשימה משתפרים ככל שמוזנות לה דוגמאות. המטרה אינה לזכור את האימון — אלא **להכליל** לדוגמאות חדשות.

שפת היסוד

- **Instance:** תצפית אחת.
- **Feature:** מאפיין של הדוגמה.
- **Feature vector:** הייצוג $x=(x_1,\dots,x^D)$.
- **Label:** הפלט הידוע בדוגמה מתויגת.
- **Model:** החוק שנלמד מהנתונים.
- **Generalization:** הצלחה על מידע שלא נראה באימון.

סוגי למידה

- **Classification:** חיזוי מחלקה בדידה.
- **Regression:** חיזוי ערך מספרי.
- **Unsupervised:** גילוי מבנה ללא תוויות.
- **Reinforcement:** סוכן משתפר באמצעות חיזוקים מהסביבה.

צינור העבודה הנכון

1. מגדירים קלט, פלט ומשימה.
2. אוספים ומייצגים דוגמאות באמצעות מאפיינים.
3. מנקים, משלימים ומנרמלים נתונים.
4. מאמנים על Training Set.
5. מכוונים Hyperparameters על Validation Set.
6. מדווחים פעם אחת על Test Set.

מלכודת: שימוש ב־Test לבחירת מודל הופך אותו לחלק מתהליך האימון; הוא כבר אינו הערכה נקייה של מידע חדש.

Preprocessing

ייצוג הנתונים קובע מה האלגוריתם מסוגל לראות. מאפיין בסולם גדול עלול להשתלט על חישובי מרחק, נתונים חסרים או רועשים עלולים לעוות דפוסים, ומספר ממדים גדול עלול לייקר את הלמידה.

Min–Max: $x' = (x - \min)/(\max - \min)$

Z-score: $x' = (x - \mu)/\sigma$

הכללה והערכת ביצועים

Underfitting

המודל פשוט מדי. גם ביצועי האימון חלשים.

מאפיין: Bias גבוה.

Overfitting

המודל מתאים את עצמו לרעש ולפרטים מקריים באימון.

מאפיין: Training טוב, Test חלש; Variance גבוה.

Confusion Matrix

אמת \ חיזוי	Positive	Negative
Positive	TP	FN
Negative	FP	TN

$$\text{Accuracy} = (TP + TN)/(TP + TN + FP + FN)$$

$$\text{Recall} = TPR = TP/(TP + FN)$$

$$\text{Precision} = TP/(TP + FP)$$

$$FPR = FP/(FP + TN)$$

Recall שואל: מתוך כל החיוביים האמיתיים, כמה זוהו? **Precision** שואל: מתוך כל מה שסומן כחיובי, כמה באמת חיובי?

ROC ו־AUC

שינוי סף ההחלטה משנה יחד את TPR ואת FPR. עקומת ROC מציגה את הזוגות האלה לכל סף; AUC הוא השטח מתחת לעקומה ומשווה מסווגים על פני ספים.

איך בוחרים מדד? לפי מחיר הטעות. אם אסור לפספס חיוביים — Recall מרכזי. אם כל התראה חייבת להיות אמינה — Precision מרכזי.

Cross Validation

כאשר יש מעט נתונים, מחלקים את קבוצת האימון ל־K חלקים. בכל סבב מתאמנים על K−1 חלקים ומאמתים על החלק הנותר. ממוצע הביצועים משמש לבחירת Hyperparameters; ה־Test נשמר לסוף.

מה חייבים לדעת: לזהות את סוג הבעיה; להבדיל Train/Validation/Test; לחשב מדדים; לזהות Overfitting/Underfitting; להסביר כיצד ייצוג הנתונים משנה את הבעיה.

K-Nearest Neighbors

האינטואיציה: דוגמאות קרובות במרחב המאפיינים צפויות להשתייך לאותה מחלקה. KNN הוא Instance-Based ו־Lazy: כמעט אין אימון, ורוב החישוב נעשה בזמן החיזוי.

האלגוריתם

1. בוחרים K ומטריקת מרחק.
2. מחשבים מרחק מהדוגמה החדשה לכל דוגמת אימון.
3. ממיינים ובוחרים את K הקרובות.
4. מבצעים הצבעת רוב או הצבעה משוקללת.

משפחת Minkowski

$$d_q(i,j) = \left(\Sigma_{r=1\ldots p} |x_{ir} - x_{jr}|^q \right)^{1/q}$$

מטריקה	נוסחה	משמעות
L_1 / Manhattan	$\Sigma \lvert x_r - z_r \rvert$	סכום ההפרשים המוחלטים
L_2 / Euclidean	$\sqrt{\Sigma (x_r - z_r)^2}$	המרחק הישר
L_∞	$\max \lvert x_r - z_r \rvert$	ההפרש הגדול ביותר בין רכיבים

חשוב: מטריקות שונות עשויות לתת סדר שכנים שונה. בחירת המרחק היא חלק מהגדרת הבעיה, לא פרט טכני.

ארבע החלטות שמשנות את התוצאה

1. מטריקת מרחק
מה פירוש "קרוב"?

2. Feature weighting
האם לכל ממד אותה חשיבות?

3. Neighbor weighting
האם שכן קרוב משפיע יותר?

4. K
כמה מקומי או חלק יהיה הגבול?

Weighted vote: $w_i = 1/d_i^2$; $\text{Score}(c) = \Sigma_{i \in N_k(x),\, y_i = c} \, 1/d_i^2$

KNN — פתרון שאלות ומלכודות

השפעת K

- **K קטן:** גבול מקומי, רגיש לדוגמה בודדת ולרעש.
- **K גדול:** גבול חלק יותר; עלול לאבד מבנה מקומי ולהתקרב למחלקת הרוב הכללית.
- **1NN ו־Voronoi:** כל דוגמת אימון מגדירה אזור שבו היא השכן הקרוב ביותר.

למה חייבים נרמול?

אם מאפיין אחד נע בין 0–1 ואחר בין 0–100,000, המאפיין הגדול ישלוט במרחק. נרמול מביא סולמות לטווח משותף. עם זאת, מאפיין מנורמל עדיין יכול להיות לא רלוונטי — ולכן Feature weighting הוא החלטה נפרדת.

תבנית פתרון

1. כתוב את נקודת השאילתה ואת כל דוגמאות האימון.
2. חשב מרחקים בטבלה — לא לפי העין.
3. מיין מהקטן לגדול וסמן את K הראשונות.
4. אם אין שקלול: ספור קולות. אם יש: חשב $1/d^2$ וסכם לכל מחלקה.
5. בדוק תיקו או מרחקים שווים בגבול הבחירה.
6. כתוב החלטה ונימוק, לא רק מחלקה.

דוגמה	מחלקה	מרחק	משקל
x_1			
x_2			
…			

מלכודת מהמצגת: בדוגמה מסוימת יש שני שכנים במרחק 1 ובמחלקות שונות עבור K=1. המצגת אינה נותנת כלל הכרעה כללי. בשאלה כזו מציינים במפורש את התיקו ולא ממציאים כלל.

יתרונות

- פשוט, שקוף וקל למימוש.
- אין שלב בניית מודל מורכב.
- מתאים טבעית למספר מחלקות.

חסרונות

- שומר את כל נתוני האימון.
- חיזוי יקר: מרחק לכל דוגמה.
- רגיש לסולם, רעש וממדים מיותרים.

עצי החלטה ו־ID3

הרעיון: מסווגים דוגמה באמצעות רצף שאלות על מאפייניה. כל מסלול מהשורש לעלה הוא כלל if–then.

מבנה העץ

- **Root:** השאלה הראשונה.
- **Internal node:** בדיקת תכונה.
- **Edge:** תשובה אפשרית.
- **Leaf:** המחלקה הסופית.

למה עץ?

- ניתן להסביר את ההחלטה.
- מסוגל לייצג קשרים היררכיים.
- יכול לפתור XOR באמצעות כמה שאלות.
- קל יחסית לחיזוי לאחר שנבנה.

Entropy — כמה המחלקות מעורבבות?

$$H(X) = -\Sigma_j\, p_j \log_2 p_j$$

- קבוצה טהורה: H=0.
- חלוקה בינארית שווה: H=1.
- ככל שהקבוצה מעורבבת יותר, אי־הוודאות גדולה יותר.

Conditional Entropy ו־Information Gain

$$H(Y|X) = \Sigma_v\, (|S_v|/|S|) \cdot H(S_v)$$

$$Gain(Y|X) = H(Y) - H(Y|X)$$

המשקל $|S_v|/|S|$ הכרחי: ענף גדול משפיע יותר מענף קטן. ID3 בוחר את התכונה שמפחיתה הכי הרבה אי־ודאות.

ID3

1. חשב Entropy של התווית בצומת.
2. לכל תכונה: חלק לענפים, חשב Entropy בכל ענף ושקלל.
3. חשב Gain ובחר את המרבי.
4. חזור רקורסיבית בכל ענף שאינו טהור.
5. עצור בקבוצה טהורה או כשאין מאפיין שמסוגל להבדיל; במקרה האחרון החזר רוב.

עצי החלטה — הכללה וגיזום

מלכודת XOR: ייתכן שכל ה־Gain-ים בשורש שווים לאפס, ובכל זאת פיצול נוסף ייצור עץ מושלם. לכן הכלל "אם Gain אפס — עצור" שגוי.

למה ID3 נקרא Greedy?

בכל צומת הוא בוחר את השיפור המקומי הטוב ביותר ואינו חוזר לשנות בחירות קודמות. לכן אין הבטחה לעץ הקטן או הטוב ביותר הגלובלי.

Training לעומת Test

$$\text{Error}_{\text{train}}(h) = \#\text{ training mistakes} \,/\, |S_{\text{train}}|$$

הגדלת העץ יכולה להמשיך להקטין Training Error, אך לאחר נקודה מסוימת Test Error עלול לעלות. זהו Overfitting: העץ מתאים פרטים מקריים ורעש.

Pre-pruning

עוצרים את בניית העץ כאשר הפיצול אינו משמעותי מספיק.

Post-pruning

בונים עץ מלא ואז מקצצים תתי־עצים לפי ביצועים על Validation.

תבנית פתרון Entropy/Gain

1. ספור חיוביים ושליליים בשורש וחשב H(Y).
2. לכל תכונה, חלק לפי ערכיה.
3. בכל ענף חשב את התפלגות המחלקות ואת האנטרופיה.
4. שקלל לפי גודל הענפים לקבלת H(Y|X).
5. חסר מהאנטרופיה המקורית ובחר Gain מרבי.
6. המשך רקורסיבית בענפים שאינם טהורים.

משפט זיכרון: Entropy שואלת "כמה המחלקות מעורבבות?"; Gain שואל "כמה הפיצול עשה סדר?"

מה חייבים לדעת

- לחשב Entropy ו־Conditional Entropy משוקללת.
- לבנות לפחות שתי רמות של עץ.
- להסביר Greedy וללא Backtracking.
- להבדיל Training, Validation ו־Test.
- לזהות Overfitting ולהסביר גיזום.

Perceptron ורשתות נוירונים

נוירון: מחשב צירוף לינארי, מוסיף Bias ומפעיל פונקציית אקטיבציה.

$$z = w^T x + b \quad ; \quad a = \varphi(z)$$

Threshold / Sign

הכרעה בדידה לפי סף.

Sigmoid

$$\sigma(x)=1/(1+e^{-x})$$

פלט רציף בין 0 ל־1.

Perceptron והגבול הלינארי

- מסווג לפי הסימן של $w^T x+b$.
- יכול לייצג AND ובעיות ניתנות להפרדה לינארית.
- אינו יכול לפתור XOR בשכבה יחידה.
- משפט ההתכנסות חל כשהנתונים ניתנים להפרדה לינארית.

למה צריך MLP?

ב־XOR אין קו יחיד שמפריד בין המחלקות. כמה נוירונים עם **אקטיבציה לא־לינארית** יכולים לחבר כמה גבולות לינאריים לגבול מורכב.

מלכודת: כמה שכבות לינאריות בלי אקטיבציה לא־לינארית שקולות לשכבה לינארית אחת — ולכן עדיין אינן פותרות XOR.

Gradient Descent

$$w \leftarrow w - \eta \cdot \partial L/\partial w$$

- η קטן מדי: למידה איטית.
- η גדול מדי: דילוגים ואי־התכנסות.
- **Batch:** עדכון מכל הדוגמאות; **SGD/Incremental:** עדכון מדוגמה או קבוצה קטנה.

Forward ו־Backward

1. **Forward:** מחשבים שכבה אחר שכבה עד לציונים ול־Loss.
2. **Backward:** משתמשים בכלל השרשרת להעברת נגזרות מה־Loss לאחור.
3. מעדכנים את הפרמטרים לפי הגרדיאנטים.

Loss ו־Backpropagation

שלושה דברים שקל לבלבל

מונח	משמעות
Score	מספר גולמי שהרשת מפיקה למחלקה.
Probability	ערך מנורמל, למשל לאחר Softmax.
Loss	קנס שמשווה את התחזית לתווית האמיתית.

Multi-Class SVM Loss

$$L_i = \Sigma_{j \neq y_i} \max(0, s_j - s_{y_i} + \Delta)$$

כל מחלקה שגויה נענשת אם הציון שלה קרוב מדי לציון המחלקה הנכונה.

Softmax Loss

$$P(Y=k|x_i)=e^{s_k}/\Sigma_j e^{s_j} \quad ; \quad L_i=-\log P(Y=y_i|x_i)$$

Softmax ממיר ציונים להסתברויות; הסתברות נמוכה למחלקה הנכונה יוצרת Loss גבוה.

גרף חישובי

אם $q=xw$ ו־$a=q+b$, אז עבור $x=3, w=2, b=1$ מתקבל $q=6, a=7$. במעבר לאחור:

$$\partial a/\partial x = w = 2 \quad ; \quad \partial a/\partial w = x = 3 \quad ; \quad \partial a/\partial b = 1$$

לעדכון המודל צריך במיוחד את הגרדיאנטים של הפרמטרים, למשל dw ו־db.

קליפת הראייה: במצגות היא מוטיבציה לעיבוד היררכי בלבד. אין כאן פרק מלא על CNN, Filters או Convolution.

מה חייבים לדעת

- לחשב פלט נוירון ואקטיבציה.
- להסביר מדוע XOR דורש אי־לינאריות.
- לבצע צעד Gradient Descent.
- לחשב SVM Loss או Softmax Loss מדוגמה קטנה.
- לבצע Forward/Backward בגרף כפל וחיבור.

Support Vector Machine

הרעיון: Perceptron יכול למצוא מפרידים רבים. SVM בוחר מפריד בעל **Margin מקסימלי**, כדי לקבל פתרון יציב יותר.

$$\text{Decision boundary:}\quad w^Tx + b = 0 \quad ; \quad f(x)=\text{sign}(w^Tx+b)$$

Margin

$$\text{Distance from plane:}\quad d=|w^Tx+b|/||w||$$

$$\text{Support planes: } w^Tx+b=\pm 1 \quad \Rightarrow \quad \text{total margin } M=2/||w||$$

מקסום Margin שקול למזעור $\frac{1}{2}||w||^2$.

Hard Margin

$$\min \tfrac{1}{2}||w||^2 \quad \text{subject to} \quad y_i(w^Tx_i+b) \geq 1$$

מתאים להפרדה לינארית מלאה, אך רגיש לרעש ולחריגים.

Soft Margin

$$y_i(w^Tx_i+b) \geq 1-\xi_i \quad ; \quad \min \tfrac{1}{2}||w||^2 + C\Sigma\xi_i$$

- **C גדול:** קנס כבד על הפרות, קרוב ל־Hard Margin.
- **C קטן:** מאפשר יותר הפרות תמורת Margin רחב יותר.
- גם כשיש הפרדה לינארית, Soft Margin עשוי להכליל טוב יותר אם נקודת חריגה מעוותת את הגבול.

Support Vectors

הדוגמאות הקרובות לגבול קובעות את ה־Margin ואת הפתרון. ב־Hard Margin הן מקיימות:

$$y_i(w^Tx_i+b)=1$$

SVM — Dual, α ו־Kernel

מה־Primal ל־Dual

$$L(w,b,\alpha)=\tfrac{1}{2}||w||^2 - \Sigma\ \alpha_i[y_i(w^Tx_i+b)-1]$$

$$w = \Sigma\ \alpha_i\ y_i\ x_i$$

$$\Sigma\ \alpha_i\ y_i = 0$$

- $\alpha_i=0$: הדוגמה אינה משתתפת ישירות בבניית w.
- $\alpha_i>0$: הדוגמה היא Support Vector.
- ב־Soft Margin: $0\le\alpha_i\le C$.

סיווג באמצעות α

$$f(x)=\text{sign}(\Sigma\ \alpha_i\ y_i\ \langle x_i,x\rangle + b)$$

רק Support Vectors תורמים לסכום.

Kernel Trick

כאשר הבעיה אינה לינארית, מדמיינים מיפוי $x\to\Phi(x)$ למרחב תכונות שבו אפשר להפריד לינארית. הקרנל מחשב את המכפלה הפנימית במרחב החדש בלי לבנות את המיפוי:

$$K(x,z)=\Phi(x)^T\Phi(z)$$

$$\text{Polynomial kernel:}\quad K(x,z)=(1+x^Tz)^2$$

$$f(x)=\text{sign}(\Sigma\ \alpha_i\ y_i\ K(x_i,x)+b)$$

משפט זיכרון: Kernel הוא קיצור דרך לחישוב דמיון אחרי מיפוי תכונות — לא אלגוריתם סיווג נפרד.

אינטואיציית המעגל

פתיחת משוואת מעגל יוצרת איברים לינאריים וריבועיים. המיפוי $(x_1,x_2)\to(x_1,x_2,x_1^2+x_2^2)$ מאפשר לייצג את הגבול כמשוואה לינארית בתכונות החדשות.

מוסכמת סימן: יש שאלות המשתמשות ב־w^Tx+b ואחרות ב־w^Tx-b. שומרים על אותה מוסכמה לאורך כל הפתרון.

SVM — תבנית פתרון

1. בדוק את מוסכמת הסימן של b.
2. זהה Support Vectors לפי $\alpha_i>0$.
3. במקרה לינארי חשב $w=\Sigma\alpha_i y_i x_i$.
4. חשב או אמת b באמצעות Support Vector ותנאי השוויון.
5. במקרה Kernel חשב כל $K(x_i,x)$; אין צורך למפות ידנית אלא אם ביקשו.
6. הצג כל איבר בסכום ואת הסימן הסופי.

ארבע נוסחאות שקל לבלבל

ביטוי	משמעות
$w^Tx+b=0$	גבול ההחלטה
$w^Tx+b=\pm1$	מישורי התמיכה לאחר נרמול
$y_i(w^Tx_i+b)\geq1$	אילוץ Hard Margin לכל דוגמה
$y_i(w^Tx_i+b)=1$	שוויון עבור Support Vector שעל השוליים

בדיקת עקביות: אם הצבת Support Vector חיובי ושלילי דורשת ערכי b שונים, הנתונים בשאלה אינם עקביים. אין להצדיק אותם בכוח.

מה חייבים לדעת

- להסביר את ההבדל בין Perceptron ל־SVM.
- לחשב Margin ולהסביר את מזעור $||w||^2$.
- להבדיל Hard/Soft Margin ולהסביר את C.
- לזהות Support Vectors ולחשב w,b.
- לסווג עם Kernel פולינומי.
- להבין ברמת העיקרון את המעבר ל־Dual.

Ensemble Learning ו־AdaBoost

Ensemble: שילוב כמה מסווגים. כדי שהשילוב יעזור, המסווגים צריכים להיות טובים מניחוש אקראי וגם מגוונים — לא לטעות תמיד על אותן דוגמאות.

Majority Vote

אם שלושה מסווגים בלתי־תלויים טועים בהסתברות $p=0.3$, הרוב טועה כאשר לפחות שניים טועים:

$$P_{error} = p^3 + 3p^2(1-p) = 0.216$$

למה שילוב עשוי לעזור?

- **סטטיסטית:** כמה היפותזות סבירות מפחיתות סיכון.
- **חישובית:** אימונים שונים עשויים להגיע למינימום מקומי שונה.
- **ייצוגית:** שילוב מודלים יכול לייצג פתרון שאינו זמין למסווג בסיס יחיד.

Bagging

יוצרים מדגמי Bootstrap בדגימה עם החזרה, מאמנים מודלים עצמאיים ומאגדים תחזיות. מתאים במיוחד למודלים לא יציבים.

Boosting

אימון סדרתי: כל סבב מתמקד יותר בדוגמאות שהסבבים הקודמים טעו בהן.

AdaBoost

1. מתחילים במשקל שווה לכל הדוגמאות.
2. מאמנים Weak Classifier ומחשבים שגיאה משוקללת ε_t.
3. מחשבים את אמינות המסווג α_t.
4. מגדילים משקל טעויות, מקטינים משקל נכונות ומנרמלים.
5. משלבים את המסווגים בהצבעה משוקללת.

$$\alpha_t = \tfrac{1}{2} \ln((1-\varepsilon_t)/\varepsilon_t)$$

$$H_{final}(x)=\text{sign}(\Sigma\ \alpha_t\ h_t(x))$$

דוגמה שגויה מוכפלת ב־e^{α_t}, ודוגמה נכונה ב־$e^{-\alpha_t}$. הבחירה ב־α_t מאזנת אחרי הנרמול את מסת המשקל של הנכונות והשגויות ל־50%–50%.

Weak אינו אקראי: במסווג בינארי נדרש בדרך כלל $\varepsilon_t<0.5$. AdaBoost גם רגיש לרעש, משום שהוא ממשיך להגדיל את משקל הדוגמאות הקשות.

Cluster Analysis

Clustering: חלוקת אובייקטים לקבוצות כך שבתוך אשכול הדוגמאות דומות ובין אשכולות הן שונות. אין תוויות מראש — זו למידה בלתי־מונחית.

שלוש גישות

גישה	רעיון
Partitioning	חלוקה ל־K קבוצות לא ריקות.
Hierarchical	בניית היררכיה באמצעות איחוד או פיצול.
Density-Based	אשכולות כאזורים צפופים המחוברים זה לזה.

K-means

1. בוחרים K מרכזים התחלתיים.
2. משייכים כל נקודה למרכז הקרוב.
3. מעדכנים כל מרכז לממוצע נקודות האשכול.
4. חוזרים עד שאין שינוי בשיוכים או במרכזים.

$$J = \Sigma_{j=1\dots K} \Sigma_{x_i \in C_j} ||x_i - \mu_j||^2$$

כל אחד משני הצעדים אינו מגדיל את J, ולכן האלגוריתם מתכנס. עם זאת, אין הבטחה לאופטימום גלובלי; אתחול שונה יכול להוביל לפתרון אחר.

יתרונות

- פשוט ויעיל יחסית.
- מתאים למאגרים גדולים.
- קל לבצע איטרציה ידנית.

חסרונות

- צריך לבחור K.
- רגיש לאתחול ולחריגים.
- מתאים בעיקר לאשכולות קומפקטיים.

מלכודת בבחירת K: אי אפשר לבחור רק לפי Distortion מינימלי, כי עבור K=n כל נקודה היא אשכול והעלות אפס. מחפשים "מרפק" שבו השיפור הנוסף קטן.

Clustering — שיטות נוספות

K-medoids

המרכז הוא דוגמה אמיתית מתוך הנתונים, לא ממוצע מלאכותי. לכן השיטה עשויה להיות עמידה יותר לחריגים.

Hierarchical Clustering

- **Agglomerative:** מתחילים מכל נקודה כאשכול ומאחדים.
- **Divisive:** מתחילים מאשכול אחד ומפצלים.
- **Single Linkage:** מרחק בין שני אשכולות הוא המרחק בין הזוג הקרוב ביותר.
- התוצאה מיוצגת ב־Dendrogram.

DBSCAN

מקבל רדיוס r ו־MinPts, ומזהה אזורים צפופים בלי לקבוע מראש את מספר האשכולות.

מושג	הגדרה
Core	בסביבת הנקודה לפחות מספר הנקודות הנדרש.
Border	אינה Core אך נמצאת ליד נקודת Core.
Noise	אינה מצטרפת לאזור צפוף.
Directly Density Reachable	נקודה בסביבת Core.
Density Reachable	קיימת שרשרת של הגעה צפיפותית; התכונה כיוונית.
Density Connected	שתי נקודות נגישות מאותה נקודה משותפת.

היתרון המרכזי: DBSCAN מזהה אשכולות בצורות לא־כדוריות ומטפל ברעש. החיסרון: עדיין צריך לבחור r ו־MinPts.

תבנית חישוב Distortion

1. חשב מרכז לכל אשכול.
2. לכל נקודה חשב מרחק אוקלידי בריבוע מהמרכז.
3. חבר בתוך כל אשכול ואז בין האשכולות.

מה חייבים לדעת

- לבצע איטרציה מלאה של K-means.
- לחשב Centroid ו־Distortion.
- להסביר אתחול ומינימום מקומי.
- להבדיל Centroid מ־Medoid.
- להבדיל Agglomerative מ־Divisive.
- לזהות Core, Border ו־Noise.

השוואת האלגוריתמים

אלגוריתם	סוג	מה נלמד?	חוזקה	רגישות / מגבלה
KNN	מונחה, Lazy	אין מודל מפורש; נשמרות הדוגמאות	פשוט ומקומי	סולם, רעש, חיזוי יקר
Decision Tree	מונחה, Eager	רצף פיצולים לפי מאפיינים	פרשנות ברורה	Overfitting; Greedy
Perceptron	מונחה, לינארי	w,b	פשוט ומהיר	לא פותר XOR
MLP	מונחה, לא־לינארי	משקולות בכמה שכבות	ייצוג מורכב	אימון ו־Hyperparameters
SVM	מונחה	מפריד Margin מקסימלי	גבול יציב; Kernels	בחירת C/Kernel; מוסכמות
Ensemble	שילוב מסווגים	אוסף מודלים והצבעה	מפחית שגיאות דרך גיוון	מודלים מתואמים לא מוסיפים הרבה
K-means	לא־מונחה	מרכזים ושיוכים	פשוט ויעיל	K, אתחול, חריגים
DBSCAN	לא־מונחה	אזורים צפופים	צורות מורכבות ורעש	r, MinPts

אותו רעיון חוזר בצורות שונות

- **ייצוג:** KNN תלוי במרחק; Kernel משנה מרחב; MLP לומד ייצוג היררכי.
- **מורכבות מול הכללה:** K קטן, עץ עמוק, רשת גדולה או C גדול יכולים להתאים יותר לאימון אך להיות רגישים יותר.
- **Validation:** משמש לבחירת K, עומק/גיזום, Learning Rate, C, Kernel ומספר אשכולות.
- **רעש וחריגים:** משפיעים על כל אלגוריתם אחרת; אין מודל "טוב תמיד".

Multi-Class

One-vs-All

בונים K מסווגים, אחד לכל מחלקה מול כל היתר.

One-vs-One

בונים מסווג לכל זוג מחלקות.

$$K(K-1)/2$$

דף נוסחאות

נושא	נוסחה
Min–Max	$x' = (x - \min)/(\max - \min)$
Z-score	$x' = (x - \mu)/\sigma$
Accuracy	$(TP+TN)/(TP+TN+FP+FN)$
Recall / TPR	$TP/(TP+FN)$
Precision	$TP/(TP+FP)$
FPR	$FP/(FP+TN)$
Minkowski	$(\Sigma \lvert x_r - z_r \rvert^q)^{(1/q)}$
KNN weight	$w_i = 1/d_i^2$
Entropy	$H(X) = -\Sigma p_j \log_2 p_j$
Conditional entropy	$H(Y\lvert X) = \Sigma_v (\lvert S_v \rvert / \lvert S \rvert) H(S_v)$
Information Gain	$Gain(Y\lvert X) = H(Y) - H(Y\lvert X)$
Neuron	$z = w^T x + b$; $a = \varphi(z)$
Gradient Descent	$w \leftarrow w - \eta \partial L / \partial w$
Softmax	$P(Y=k\lvert x) = e^{s_k} / \Sigma_j e^{s_j}$
SVM margin	$M = 2/\lvert\lvert w \rvert\rvert$
SVM dual weights	$w = \Sigma \alpha_i y_i x_i$
Kernel classifier	$\text{sign}(\Sigma \alpha_i y_i K(x_i, x) + b)$
AdaBoost	$\alpha_t = \frac{1}{2} \ln((1 - \varepsilon_t)/\varepsilon_t)$
K-means objective	$J = \Sigma_j \Sigma_{x_i \in C_j} \lvert\lvert x_i - \mu_j \rvert\rvert^2$

כלל עבודה: נוסחה אינה פתרון. לפני ההצבה כותבים מה כל משתנה מייצג, ואחרי החישוב מסבירים את המשמעות.

תבניות פתרון מהירות

KNN

1. חשב מרחקים.
2. מיין.
3. בחר K.
4. הצבע/שקלל.
5. בדוק תיקו.

ID3

1. Entropy בשורש.
2. פיצולים.
3. Entropy משוקללת.
4. Gain.
5. רקורסיה.

Neural Network

1. Forward.
2. Scores/Probabilities.
3. Loss.
4. Backward.
5. Update.

SVM

1. מוסכמת b.
2. $\alpha>0$.
3. w או Kernel.
4. אימות b.
5. Sign.

AdaBoost

1. ε משוקללת.
2. α.
3. עדכון משקלים.
4. נרמול.
5. Weighted vote.

K-means

1. שיוך.
2. ממוצע.
3. חזרה.
4. עצירה.
5. Distortion.

בדיקות sanity לפני הגשה

- האם הסתברויות/משקלים שאמורים להסתכם ל־1 אכן מסתכמים?
- האם שקללת ענפים לפי גודלם ב־Conditional Entropy?
- האם השתמשת במוסכמת הסימן של שאלת SVM?
- האם מרכז K-means חושב מחדש מכל נקודות האשכול?
- האם בחרת שכנים לפי טבלה ולא לפי ציור בלבד?
- האם כתבת נימוק ולא רק מספר סופי?

צ׳קליסט למבחן

יסודות

- Classification / Regression / Unsupervised
- Train / Validation / Test
- Overfitting / Underfitting
- Precision / Recall / FPR

KNN

- L_1, L_2, L_∞
- Normalization
- Feature vs Neighbor weighting
- השפעת K ותיקו

Decision Trees

- Entropy ו־Gain
- ממוצע משוקלל
- ID3 רקורסיבי
- XOR ו־Pruning

Neural Networks

- Neuron + Activation
- XOR ואי־לינאריות
- Gradient Descent
- Loss + Backprop

SVM

- Margin ו־Support Vectors
- Hard/Soft ו־C
- α, w, b
- Kernel classifier

Ensemble

- דיוק + גיוון
- Majority Vote
- Bagging vs Boosting
- ε, α ועדכון משקלים

Clustering

- איטרציית K-means
- Distortion
- Centroid vs Medoid
- Core / Border / Noise

הרגל פתרון

- לכתוב נתונים וסימונים
- להציג שלבים
- לבדוק יחידות/טווחים
- להסביר את התוצאה

הקו שמחבר את כל הקורס: מהו ייצוג הקלט, מה האלגוריתם לומד, כיצד הוא חוזה דוגמה חדשה, ומה מבטיח שהביצועים מכלילים מעבר לקבוצת האימון?

היקף: מבוא, KNN, עצי החלטה, Perceptron/MLP, Gradient Descent, Loss, Backpropagation, SVM, Ensemble/AdaBoost ו־Cluster Analysis. תרגול 13 שולב בפרקי SVM, AdaBoost ו־Naive Bayes. אינו כלול בהתאם למפת הקורס.